

# PRACTICAL WEEK – 11

TASK1: Develop a multithreaded counter application using POSIX threads (pthread\_create, pthread\_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

```
shreyaj@LAPTOP-FT6VVM4D: ~$ mkdir -p ~/Practical11
cd ~/Practical11
shreyaj@LAPTOP-FT6VVM4D:~/Practical11$ nano race.c
shreyaj@LAPTOP-FT6VVM4D:~/Practical11$ gcc race.c -o race -pthread
./race
Initial counter value: 0
Expected counter value: 500000
Actual counter value: 100200
shreyaj@LAPTOP-FT6VVM4D:~/Practical11$ |
```

```
GNU nano 0.7.1 race.c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_THREADS 5
#define INCREMENTS 100000

int counter = 0;

void *increment_counter(void *arg) {
    for (int i = 0; i < INCREMENTS; i++) {
        int temp = counter;
        usleep(1);
        counter = temp + 1;
    }
    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];

    printf("Initial counter value: %d\n", counter);

    // Create 5 threads
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_create(&threads[i], NULL, increment_counter, NULL);
    }

    // Wait for all threads to finish
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Expected counter value: %d\n", NUM_THREADS * INCREMENTS);
    printf("Actual counter value: %d\n", counter);

    return 0;
}
```

TASK2: Modify the previous program using mutex locks (pthread\_mutex\_lock, pthread\_mutex\_unlock) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

```
shreyaj@LAPTOP-FT6VVM4D:~/Practical11$ nano mutex.c
shreyaj@LAPTOP-FT6VVM4D:~/Practical11$ gcc mutex.c -o mutex -pthread
./mutex
Initial counter value: 0
Expected counter value: 500000
Actual counter value: 500000
shreyaj@LAPTOP-FT6VVM4D:~/Practical11$ |
```

```
shreyaj@LAPTOP-FT6VVM4D: x + v
GNU nano 8.7.1 mutex.c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_THREADS 5
#define INCREMENTS 100000

int counter = 0;
pthread_mutex_t lock;

void *increment_counter(void *arg) {
    for (int i = 0; i < INCREMENTS; i++) {
        pthread_mutex_lock(&lock);
        int temp = counter;
        usleep(1);
        counter = temp + 1;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];

    // Initialize the mutex lock
    pthread_mutex_init(&lock, NULL);

    printf("Initial counter value: %d\n", counter);

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_create(&threads[i], NULL, increment_counter, NULL);
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Expected counter value: %d\n", NUM_THREADS * INCREMENTS);
    printf("Actual counter value: %d\n", counter);

    // Clean up mutex resources
    pthread_mutex_destroy(&lock);
}
```

```
int main() {
    pthread_t threads[NUM_THREADS];

    // Initialize the mutex lock
    pthread_mutex_init(&lock, NULL);

    printf("Initial counter value: %d\n", counter);

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_create(&threads[i], NULL, increment_counter, NULL);
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Expected counter value: %d\n", NUM_THREADS * INCREMENTS);
    printf("Actual counter value: %d\n", counter);

    // Clean up mutex resources
    pthread_mutex_destroy(&lock);

    return 0;
}
```